



RedShell

Advanced Framework

USER GUIDE

Setup, walkthrough, and demo guide for your Final Year Project

A unified security assessment platform: network recon, CVE intelligence, web application scanning, live host monitoring, and AI-assisted analysis.

Generated for local, authorized use only.

Contents

1. What This Is
2. Before You Start — Authorized Use Only
3. Requirements
4. Installing & Running on Windows (Recommended)
5. Installing & Running on Linux / macOS
6. Manual Setup (Any OS)
7. First Launch Checklist
8. Using the Dashboard — Overview Tab
9. Using the Dashboard — Recon Tab
10. Using the Dashboard — CVE Intel Tab
11. Using the Dashboard — Web Scan Tab
12. Using the Dashboard — Live Monitor Tab
13. Using the Dashboard — AI Analyst Tab
14. Using the Dashboard — History Tab
15. Enabling the AI Analyst (Optional)
16. Recommended Demo Flow for Your Viva
17. Troubleshooting
18. Project File Structure Reference
19. Legal & Ethical Notes

1. What This Is

RedShell Advanced Framework is a unified, browser-based security assessment platform you built for your Final Year Project. It combines five capabilities behind one dashboard:

- **Recon** — concurrent TCP port scanning with service fingerprinting and banner grabbing
- **CVE Intelligence** — correlates discovered services against the NVD vulnerability database
- **Web Scanner** — detection-only checks for missing security headers, insecure cookies, and reflected-input / SQL-error indicators
- **Live Monitor** — real-time host telemetry with rule-based anomaly detection
- **AI Analyst** — Claude-powered risk scoring and plain-English summaries, with an offline rule-based fallback

Everything runs locally on your own machine. No data leaves your computer except optional, explicit calls to the NVD public API (for CVE lookups) and the Anthropic API (only if you choose to enable the AI Analyst).

2. Before You Start — Authorized Use Only

■ Read this before running any scan

RedShell is built strictly for authorized security testing — against systems you own, or have explicit written permission to test. The project ships with a safe, local demo target (`demo_target/vulnerable_app.py`) specifically so you never need real infrastructure to test every feature. Do not point any module at systems you do not own or are not authorized to test. Unauthorized scanning is illegal in most jurisdictions (e.g. the Computer Fraud and Abuse Act in the US, the Computer Misuse Act in the UK, PECA 2016 in Pakistan). All web-scanning checks are detection-only: they identify potential weaknesses for a human to manually verify. They do not exploit, extract real data, or modify target state.

Stick to **127.0.0.1 / localhost** and the bundled demo target unless you have your own lab VM or written authorization. This is also the safest way to demo the tool to your supervisor or examiner.

3. Requirements

Requirement	Details
Operating System	Windows 10/11 (primary), or Linux / macOS
Python	Version 3.10 or newer (3.11 / 3.12 recommended)
Internet (optional)	Only needed for live NVD CVE lookups and live AI analysis — both have offline fallbacks
Browser	Any modern browser (Chrome, Edge, Firefox)
Disk space	~150 MB for the virtual environment and dependencies

Don't have Python yet? Download it from python.org/downloads. On the Windows installer, make sure to tick **"Add Python to PATH"** — this is the single most common setup mistake.

4. Installing & Running on Windows (Recommended)

This is the easiest path — one script does everything.

1. Install Python

Go to python.org/downloads, download the latest Python 3.x installer, run it, and **tick "Add Python to PATH"** before clicking Install.

2. Unzip the project

Extract RedShell-Advanced-Framework.zip anywhere convenient, e.g. Desktop\redshell.

3. Run the launcher

Double-click **run_windows.bat** inside the redshell folder.

4. Wait for first-run setup

The first run creates a virtual environment and installs dependencies automatically — this can take 1-3 minutes. You'll see progress messages in the command window.

5. Browser opens automatically

Once both servers are up, your default browser opens to **http://localhost:8000** — the RedShell dashboard.

If Windows SmartScreen warns you

Since this is an unsigned local script, Windows may show a SmartScreen warning the first time. Click **"More info" → "Run anyway"**. This is expected for any locally-written .bat script and is safe to do since you (or your supervisor) can read every line of run_windows.bat — it's plain text.

To stop RedShell later, just close the command window, or press **Ctrl+C** inside it.

5. Installing & Running on Linux / macOS

Open a terminal in the project folder and run:

```
chmod +x run_linux_mac.sh
./run_linux_mac.sh
```

Just like the Windows script, this creates a virtual environment on first run, installs dependencies, starts the demo target and the dashboard, and attempts to open your browser automatically. If the browser doesn't open, manually visit **<http://localhost:8000>**.

Press **Ctrl+C** in the terminal to stop RedShell.

6. Manual Setup (Any OS)

If you prefer full control, or the launcher script has an issue, here's the equivalent done by hand. You'll need two terminal windows.

Step 1 — Create and activate a virtual environment:

```
python -m venv venv

# Windows:
venv\Scripts\activate

# Linux / macOS:
source venv/bin/activate
```

Step 2 — Install dependencies:

```
pip install -r backend/requirements.txt
pip install flask
```

Step 3 — Terminal window 1: start the demo target

```
python demo_target/vulnerable_app.py
```

Step 4 — Terminal window 2: start the RedShell backend

```
cd backend
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Step 5 — Open your browser to:

```
http://localhost:8000
```


7. First Launch Checklist

Once running, confirm these three things before you start exploring:

✓	Dashboard loads	http://localhost:8000 shows the RedShell dashboard with a dark sidebar
✓	Demo target is up	http://localhost:5050 in a separate tab shows "RedShell Demo Target"
✓	Status pill shows	Top-right of the dashboard shows either "AI Analyst Online" or "AI Offline (rule-based fallback)" — both are normal

If any of these fail, jump to Section 17 (Troubleshooting).

8. Using the Dashboard — Overview Tab

What it does

The Overview tab runs the **full pipeline** in one go: recon → CVE correlation → web scan → AI risk analysis, all streamed live via WebSocket so you can watch each stage execute in real time. This is the tab you'll use for your main demo.

Default field value

```
Host/IP = 127.0.0.1
```

How to use it

- Enter a Host/IP (e.g. 127.0.0.1) and a target URL (e.g. http://127.0.0.1:5050)
- Tick or untick which modules to include: CVE correlation, Web scan, AI analysis
- Click **"Run Full Assessment"**
- Watch the animated port-sweep panel and the live console log as each stage completes

What you'll see

A progress bar with stage labels (recon → cve → web → ai → done), a live console log, then consolidated result panels: AI risk summary, recon port table, matched CVE cards, and web scan findings.

9. Using the Dashboard — Recon Tab

What it does

Runs a standalone network reconnaissance scan: a concurrent TCP connect scan across ~39 common ports, with service identification, banner grabbing, TLS detection on relevant ports, and a lightweight OS fingerprint guess based on the open-port signature.

Default field value

```
Host/IP = 127.0.0.1
```

How to use it

- Enter a Host/IP to scan
- Click "**Scan Ports**"
- Results appear within a few seconds (the scan is concurrent, not sequential)

What you'll see

A summary row (open port count, scan duration, resolved IP), an OS fingerprint guess, and a table of open ports with service name, transport (plain/TLS), banner text, and response latency.

10. Using the Dashboard — CVE Intel Tab

What it does

Runs recon, then takes every detected service and queries the live NVD (National Vulnerability Database) API for matching CVEs. If the NVD API is unreachable or rate-limited, it automatically falls back to a small bundled offline dataset covering common services (SSH, SMB, RDP, MySQL, Redis, etc.) so the feature still works without internet.

Default field value

```
Host/IP = 127.0.0.1
```

How to use it

- Enter a Host/IP
- Click "**Scan & Correlate**"
- This can take 10–30 seconds depending on how many services are found — the NVD public API is rate-limited, so RedShell paces its requests politely

What you'll see

A list of CVE cards, each showing the CVE ID, severity badge, CVSS score, matched service/port, and a short summary. Cards are sorted by severity, most critical first.

11. Using the Dashboard — Web Scan Tab

What it does

Runs detection-only web application checks: missing security headers (CSP, HSTS, X-Frame-Options, etc.), insecure cookie flags (HttpOnly/Secure/SameSite), missing HTTPS enforcement, and two safe detection probes — reflected-input detection and SQL-error-disclosure detection — using inert canary strings rather than real exploit payloads.

Default field value

Target URL = `http://127.0.0.1:5050`

How to use it

- Enter a target URL — try these specific demo-target paths for the most interesting results:
- `http://127.0.0.1:5050/` — header and cookie findings
- `http://127.0.0.1:5050/search` — reflected-input finding
- `http://127.0.0.1:5050/product` — SQL-error-disclosure finding
- Click **"Scan Application"**

What you'll see

Severity count chips (Critical/High/Medium/Low/Info) followed by a list of finding cards, each with a severity badge, category, description, evidence (e.g. the exact test URL used), and a remediation tip.

12. Using the Dashboard — Live Monitor Tab

What it does

Watches the machine RedShell itself is running on: CPU, memory, disk usage, listening ports, and established connections. Applies simple rule-based and statistical anomaly detection — for example, flagging a sudden connection-count spike that could indicate scanning activity, or a sustained CPU spike that could indicate a runaway process.

How to use it

- Click **"Start Monitoring"** — a green pulsing dot confirms it's live
- Gauges update every ~3 seconds
- To generate a visible anomaly for a demo: open Task Manager / Activity Monitor and run something CPU-intensive briefly, or open several new network connections (e.g. a few browser tabs to different sites) — watch for an alert to appear in the feed
- Click **"Stop"** to end monitoring, or **"Clear Alerts"** to reset the feed

What you'll see

Four live gauges (CPU %, Memory %, Disk %, plus connection/port counts) and a scrolling alert feed with severity-colored entries and timestamps.

13. Using the Dashboard — AI Analyst Tab

What it does

A chat interface to ask follow-up questions about your most recent scan results — e.g. "what should I fix first?" or "explain the SQLi finding". Powered by the Claude API when an API key is configured server-side (see Section 15); otherwise it tells you AI chat requires a key to be set.

How to use it

- Run a Full Assessment first (Overview tab) so there's scan data to discuss
- Type a question in the chat box and press Enter or click Send

What you'll see

A conversational reply grounded in your actual scan data — not a generic answer.

14. Using the Dashboard — History Tab

What it does

Every Full Assessment is automatically saved to a local SQLite database (data/redshell.db), so you can revisit past scans without re-running them — useful for comparing results across a demo or building screenshots for your report.

How to use it

- Click the History tab to load the list of past scans
- Click any entry to reopen its full results in the Overview tab, exactly as they appeared originally

What you'll see

A list of past assessments with target, timestamp, and scan type, newest first.

15. Enabling the AI Analyst (Optional)

RedShell works fully without this step — the AI Analyst and the AI Risk Analysis panel both fall back to a deterministic, rule-based summarizer if no API key is configured. This means your demo **never breaks** due to a missing or expired key. Enabling real AI analysis is optional polish.

1. Get an API key

Visit console.anthropic.com and create an API key.

2. Create your .env file

In the project root, copy `.env.example` to a new file named exactly `.env`

3. Add your key

Open `.env` in a text editor and fill in:

```
ANTHROPIC_API_KEY=your_key_here
```

4. Restart RedShell

Close and re-run the launcher script.

Confirm it worked: the pill in the top-right of the dashboard should now read **"AI Analyst Online"** instead of **"AI Offline (rule-based fallback)"**.

Keep your key private

Never commit your `.env` file to GitHub or share it publicly — it's already excluded via `.gitignore`. Treat your API key like a password.

16. Recommended Demo Flow for Your Viva

A suggested 5-minute walkthrough that shows every module working together, using only the bundled demo target — no external systems or internet dependency required.

1. Open the Overview tab

Set Host/IP to 127.0.0.1 and URL to http://127.0.0.1:5050. Leave all three checkboxes ticked.

2. Run the Full Assessment

Click Run. Narrate the live port sweep and console log as recon executes — point out it found the demo target's port (5050) and your own backend's port (8000).

3. Walk through CVE matches

Explain that RedShell correlates discovered services against NVD, with an offline fallback if there's no internet — a deliberate resilience decision.

4. Walk through web findings

Point out the missing-header findings, the insecure cookie finding, and explain that the reflected-input and SQL-error checks are detection-only by design — a DAST-style approach, the same category as tools like OWASP ZAP.

5. Show the AI risk summary

Read the plain-English executive summary and prioritized fix list. If AI is offline, explain the rule-based fallback is intentional engineering, not a missing feature.

6. Switch to Live Monitor

Click Start Monitoring. Briefly open a few extra connections or run something CPU-heavy to trigger a live alert.

7. Open AI Analyst

Ask a question like "what should I fix first?" and show the grounded answer.

8. Open History

Click your scan to show it persisted correctly — demonstrates the full data lifecycle: scan → store → retrieve → display.

17. Troubleshooting

Symptom	Likely Cause	Fix
"python is not recognized"	Python not on PATH	Reinstall Python and tick "Add Python to PATH", or restart your terminal/PC after installing
pip install fails on Linux	System Python is externally managed	Make sure you activated the venv first (source venv/bin/activate) before running pip
Browser doesn't open automatically	Default browser config / OS quirk	Manually navigate to http://localhost:8000
"Address already in use" on port 8000 or 5050	A previous RedShell instance is still running	Close any old command windows running RedShell, or restart your PC, then relaunch
Web Scan tab shows "Could not connect"	Demo target isn't running	Confirm http://127.0.0.1:5050 loads in a browser tab; if not, the demo target process may have stopped — relaunch via the script
CVE Intel tab is slow	Live NVD API rate limiting (expected)	This is normal — NVD's public API allows ~5 requests per 30 seconds without a key; RedShell paces requests automatically
AI Analyst says key is required	No .env / ANTHROPIC_API_KEY set	This is expected without a key — see Section 15. The rest of the app works fully without it
Windows shows a SmartScreen warning	Unsigned local .bat script (expected)	Click "More info" → "Run anyway" — see Section 4
Live Monitor shows permission errors	OS restricts connection listing for some processes	This is normal on some systems; CPU/memory gauges will still work correctly

Still stuck? Check the command window output for the actual Python error message — it almost always points directly at the problem (e.g. a missing dependency or a port conflict).

18. Project File Structure Reference

```
redshell/  
  ■■■ backend/  
  ■ ■■■ main.py FastAPI app: REST + WebSocket routes  
  ■ ■■■ database.py SQLite persistence for scan history  
  ■ ■■■ requirements.txt  
  ■ ■■■ modules/  
  ■ ■■■ recon.py Port scanning & service fingerprinting  
  ■ ■■■ cve_lookup.py NVD API correlation + offline fallback  
  ■ ■■■ web_scanner.py Detection-only web app security checks  
  ■ ■■■ monitor.py Host-based anomaly detection  
  ■ ■■■ ai_analyst.py Claude API integration + fallback  
  ■■■ frontend/  
  ■ ■■■ index.html Dashboard structure + design system  
  ■ ■■■ app.js Dashboard logic, WebSocket handling  
  ■■■ demo_target/  
  ■ ■■■ vulnerable_app.py Safe, local, intentionally vulnerable app  
  ■■■ data/ SQLite database (created on first run)  
  ■■■ run_windows.bat One-click Windows launcher  
  ■■■ run_linux_mac.sh One-click Linux/macOS launcher  
  ■■■ .env.example API key template  
  ■■■ README.md Full technical documentation
```

The README.md in the project root contains the same setup instructions in Markdown form, plus deeper architectural notes — useful to reference directly in your written report.

19. Legal & Ethical Notes

Consider including a version of this statement in your project report — it shows your examiners you've thought about the ethics of building security tooling, not just the implementation:

"RedShell is designed around the principle of authorized, non-destructive assessment. All web application checks are detection-only: they identify indicators of potential vulnerabilities (e.g. unescaped reflected input, database error signatures) without exploiting them or extracting real data. This design choice reflects real-world DAST (Dynamic Application Security Testing) tool behavior — such as that used by OWASP ZAP or Burp Suite's passive scanner — and was a deliberate decision to keep the framework usable for ethical, authorized security testing only."

Good luck with your testing tomorrow and your final submission. The project is built to run cleanly out of the box — if anything doesn't behave as described here, Section 17 is your first stop.